
1 innerHTML

♦ What it does

- **Parses HTML**
- Reads or writes **raw HTML markup**
- Triggers the **HTML parser**

♦ Example

```
<div id="e1"><span>Hello</span></div>
```

```
e1.innerHTML
```

```
// "<span>Hello</span>"
```

```
e1.innerHTML = "<b>Hi</b>"
```

👉 Browser:

- Destroys old children
- Parses new HTML
- Creates **new DOM nodes**

♦ Key Properties

Feature	innerHTML
Returns HTML	✓
Parses tags	✓

Executes parser	✓
Preserves markup	✓
XSS risk	⚠ Yes

◆ Performance

- **Slowest**
- Requires:
 - HTML parsing
 - DOM creation
 - Layout recalculation

◆ Security ⚠

`e1.innerHTML = userInput // ❌ XSS risk`

2 innerText

◆ What it does

- Returns **human-visible text**
- Respects **CSS styles**
- Triggers **layout & reflow**

◆ Example

```
<div>
  Hello
  <span style="display:none">Hidden</span>
```

```
</div>
```

```
el.innerText
```

```
// "Hello"
```

◆ Key Properties

Feature	innerText
Returns visible text	✓
Reads CSS	✓
Triggers reflow	✗ Slow
Removes hidden text	✓
Normalizes whitespace	✓

◆ Internals 🧠

- Browser must:
 - Compute styles
 - Check visibility
 - Perform layout

That's why it's **slow**.

3 textContent

◆ What it does

- Returns **raw text**

- Ignores CSS
- **No layout calculation**

♦ Example

```
<div>
  Hello
  <span style="display:none">Hidden</span>
</div>
```

```
el.textContent
// "\n Hello\n Hidden\n"
```

♦ Key Properties

Feature	textContent
Returns all text	✓
Ignores CSS	✓
Preserves whitespace	✓
Triggers reflow	✗
Fast	🚀

♦ Performance

- **Fastest**
- Direct DOM tree traversal

4 Side-by-side comparison

Feature	innerHTML	innerText	textContent
---------	-----------	-----------	-------------

Parses HTML	✓	✗	✗
Returns HTML	✓	✗	✗
Returns visible text	✗	✓	✗
Returns hidden text	✗	✗	✓
Respects CSS	✗	✓	✗
Preserves whitespace	✗	✗	✓
Triggers reflow	✗	✓	✗
Performance	Slow	Slowest	Fastest
XSS risk	High	Low	Low

5 Writing behavior (very important)

```
e1.innerHTML = "<span>Hi</span>" // creates DOM nodes
e1.innerText = "<span>Hi</span>" // shows literal text
e1.textContent = "<span>Hi</span>" // shows literal text
```

Result on screen:

```
<span>Hi</span>
```

Only `innerHTML` creates elements.

6 Which one should YOU use?

✓ Use `textContent` (most cases)

- Safe

- Fast
- Predictable

```
e1.textContent = userInput
```

⚠️ Use **innerHTML** only if

- You **trust** the source
- You need HTML rendering

```
e1.innerHTML = trustedHTML
```

❌ Avoid **innerText** unless

- You need **what user sees**
- You care about CSS visibility

7 Internals (browser-engine level 🧠)

`innerHTML` → HTML Parser → DOM Tree → Layout

`innerText` → DOM → Style → Layout → Visible Text

`textContent` → DOM → Text Nodes

That's why:

- `textContent` is fastest
 - `innerText` is slowest
-

8 Interview one-liner (gold)

`textContent` is raw text, `innerText` is rendered text, `innerHTML` is parsed markup.

1 What are HTMLCollection and NodeList?

Both are **array-like objects** returned by DOM APIs when you query the document.

They are **NOT** real arrays.

◆ HTMLCollection

What it is

- A live collection of **ELEMENT** nodes only
- Updates automatically when DOM changes

Returned by

```
document.getElementsByTagName("div")
document.getElementsByClassName("box")
element.children
```

Example

```
const divs = document.getElementsByTagName("div")
```

If a new `<div>` is added → `divs` updates automatically.

Key Properties

- `.length`
- `.item(index)`
- `.namedItem(idOrName)`

✗ No `map`, `filter`, `forEach`

◆ NodeList

What it is

- A list of **DOM nodes**
- Can contain:
 - Elements
 - Text nodes
 - Comments

Returned by

```
document.querySelectorAll("div")  
element.childNodes
```

Two types of NodeList

1 Static NodeList (most common)

```
document.querySelectorAll("div")
```

- Snapshot at query time
- **Does NOT update** if DOM changes

2 Live NodeList (rare)

`element.childNodes`

2 Core difference (LIVE vs STATIC)

🔥 THIS IS THE MOST IMPORTANT PART

```
const live = document.getElementsByTagName("div")
const static = document.querySelectorAll("div")
```

```
document.body.appendChild(document.createElement("div"))
```

```
live.length    // ✅ increased
static.length  // ❌ unchanged
```

Why?

- `HTMLCollection` is **live**
 - `querySelectorAll` returns **static NodeList**
-

3 Node types difference

```
element.children  // HTMLCollection (elements only)
element.childNodes // NodeList (text + comments + elements)
```

```
<div>
  Hello
  <span></span>
</div>
```

```
children.length    // 1
childNodes.length  // 3 (text, span, text)
```

4 Comparison table (clear & exact)

Feature	HTMLCollection	NodeList
Live	✓ Always	✗ Mostly static
Contains elements only	✓	✗
Contains text nodes	✗	✓
Returned by	getElements*	querySelectorAll
forEach	✗	✓ (static only)
map/filter	✗	✗
Array methods	✗	✗

5 Why these exist (design reason)

Historical reason

- **HTMLCollection** came before ES5 arrays
- Designed for **dynamic DOM updates**

Modern DOM

- **NodeList** introduced for **CSS selector API**
 - Safer snapshots (**querySelectorAll**)
-

6 Why convert to Array? (VERY IMPORTANT)

🔥 Reason 1: Array methods

```
Array.from(nodeList).map(...)
```

Neither supports:

- `map`
 - `filter`
 - `reduce`
-

🔥 Reason 2: Avoid live collection bugs

```
const items = document.getElementsByClassName("item")
```

```
for (let i = 0; i < items.length; i++) {  
  items[i].remove()  
}
```

💣 BUG: length keeps changing → elements skipped

```
const items = Array.from(document.getElementsByClassName("item"))  
items.forEach(el => el.remove())
```

✅ Safe

🔥 Reason 3: Predictability

Live collections:

- Change **behind your back**
- Break loops
- Cause infinite loops

Arrays:

- Stable snapshot
- Deterministic behavior

Reason 4: Performance clarity

`Array.from(nodeList) // O(n) once`

Better than:

- Recalculating live DOM on each access
-

7 How to convert (best ways)

NodeList / HTMLCollection → Array

`const arr = Array.from(collection)`

or

`const arr = [...collection]`

8 When NOT to convert

✗ If:

- You only need `.length`
- You want live updates

```
const live = document.getElementsByTagName("li")
```

9 Interview gold answer 🏆

HTMLCollection is a live list of elements, NodeList is usually a static list of nodes. We convert them to arrays for predictability, safety, and array methods.



Mental model (remember this)

HTMLCollection → Live → Elements only

NodeList → Static → All node types

Array → Stable → Functional operations